Python Bytecode, .pyc Caching and Execution

Study notes in easy words: bytecode, .pyc files, caching, the PVM, CPython and interview questions.

Big picture

Python compiles your code into bytecode, the PVM runs that bytecode, and only imported modules get their bytecode saved to disk.

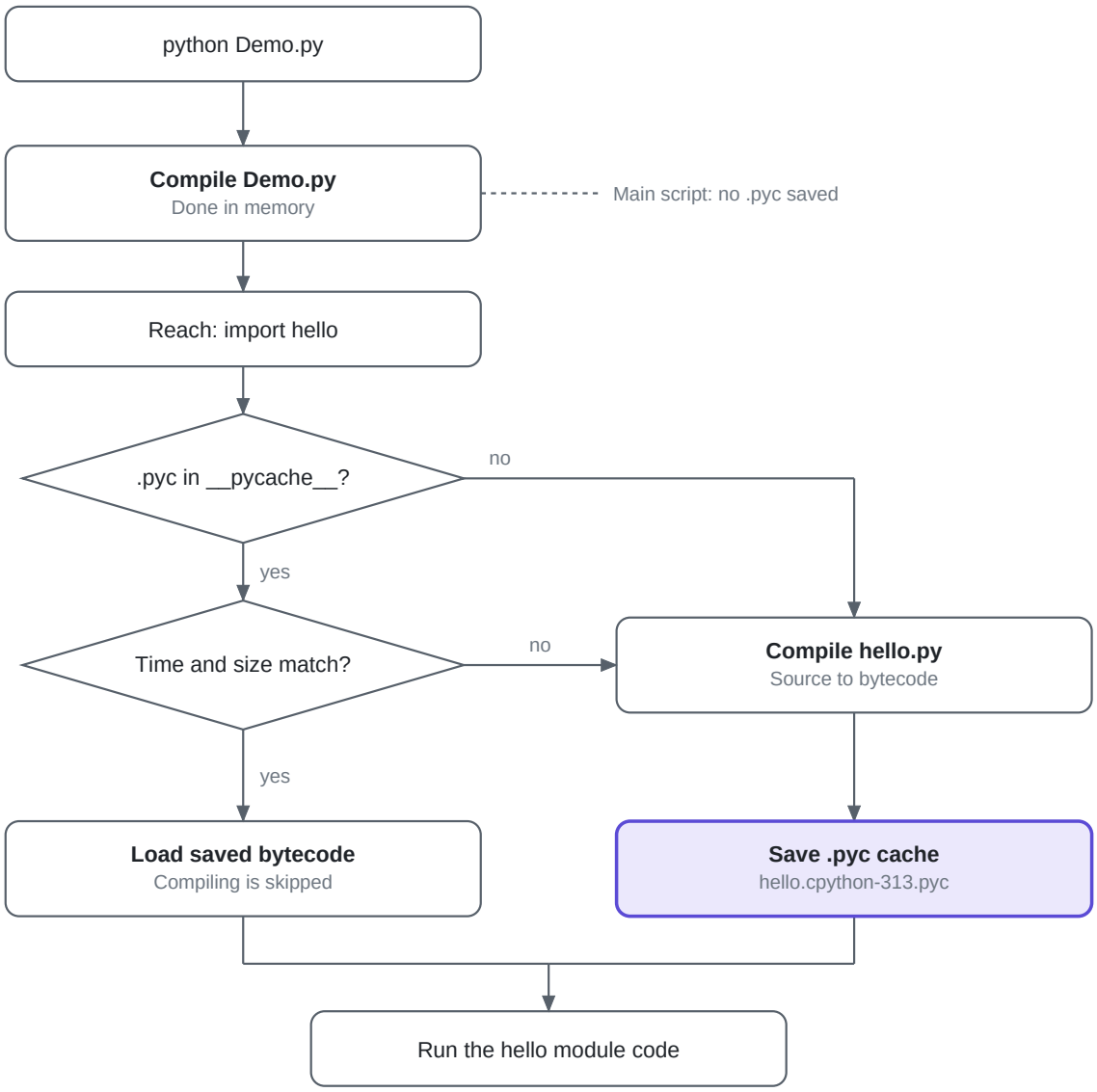
1. Source code (`.py`) is compiled into bytecode, in memory.
2. An imported module's bytecode is saved as a `.pyc` file in `__pycache__`. The main script's bytecode is not saved.
3. On later runs, a valid .pyc is loaded and compiling is skipped.
4. The PVM executes the bytecode, one instruction at a time.
5. The CPU executes the PVM's machine code.
6. The output appears.

Key terms

Term	Meaning
Source code (`.py`)	The code you write. Humans can read it.
Bytecode	Low-level instructions made from your source code, such as `LOAD_CONST` and `CALL`.
`.pyc` file	A file that stores saved bytecode, kept in the `__pycache__` folder (example: `hello.cpython-313.pyc`, where 313 means Python 3.13).
Caching	Saving the result of some work so it can be reused instead of repeated. Pronounced "cash" (caching = "CASH-ing").
PVM (Python Virtual Machine)	The part of Python that reads bytecode and executes it, one instruction at a time.
CPython	The standard, official Python (from python.org), written in the C language. It contains the compiler and the PVM.

Flowchart 1: compile and cache (Demo.py imports hello)

Only an imported module gets a saved .pyc



Read it top to bottom: the main script is never saved, and an imported module is compiled and saved only when no valid .pyc exists.

What gets saved and what does not

Bytecode is made for every file that runs, but a `.pyc` is saved only for files that are imported.

File	How it is used	Bytecode made?	.pyc saved?
abc.py	Run directly	Yes, in memory	No
abc.py	Imported by hij.py	Yes	Yes, in `__pycache__`
hij.py	Run directly	Yes, in memory	No
hij.py	Imported by main.py	Yes	Yes

Example: you run `python Demo.py` and it does `import hello`. `Demo.py` is compiled in memory and not saved. `hello.py` is compiled and its .pyc is saved.

When Python reuses the .pyc

Python saves a module's .pyc once and reuses it until the source file changes.

1. Python looks for the .pyc in __pycache__.
2. The .pyc stores the source file's last-modified time and size.
3. If they match the current .py file, Python loads the saved bytecode and skips compiling.
4. If you edited the .py file, they do not match. Python recompiles and overwrites the .pyc.

You can create a .pyc without importing, with `python -m py_compile abc.py`.

Why the main script is not saved

.pyc caching belongs to the import system, and a main script gains almost nothing from it. The Python docs state the behavior but do not record the developers' exact reasoning, so these are the practical reasons:

1. A script run directly does not go through import, so the caching step never runs.
2. The main script is usually small, so compiling it takes almost no time.
3. Modules hold most of the code and are loaded again and again, so caching them saves real time.
4. Saving a .pyc for every script would create extra files, even in read-only or shared folders.

Common mistake	Correct idea
The main script runs only once, so it is not saved.	A program can be opened every day. The script is skipped because it is small and not imported.
The .pyc is saved again and again.	It is saved once, and again only if the source file changes.
A .pyc makes the program run faster.	It makes the program load faster. Execution speed is the same.

Who runs the bytecode

The PVM runs the bytecode, the CPU runs the PVM, and CPython never converts bytecode into machine code.

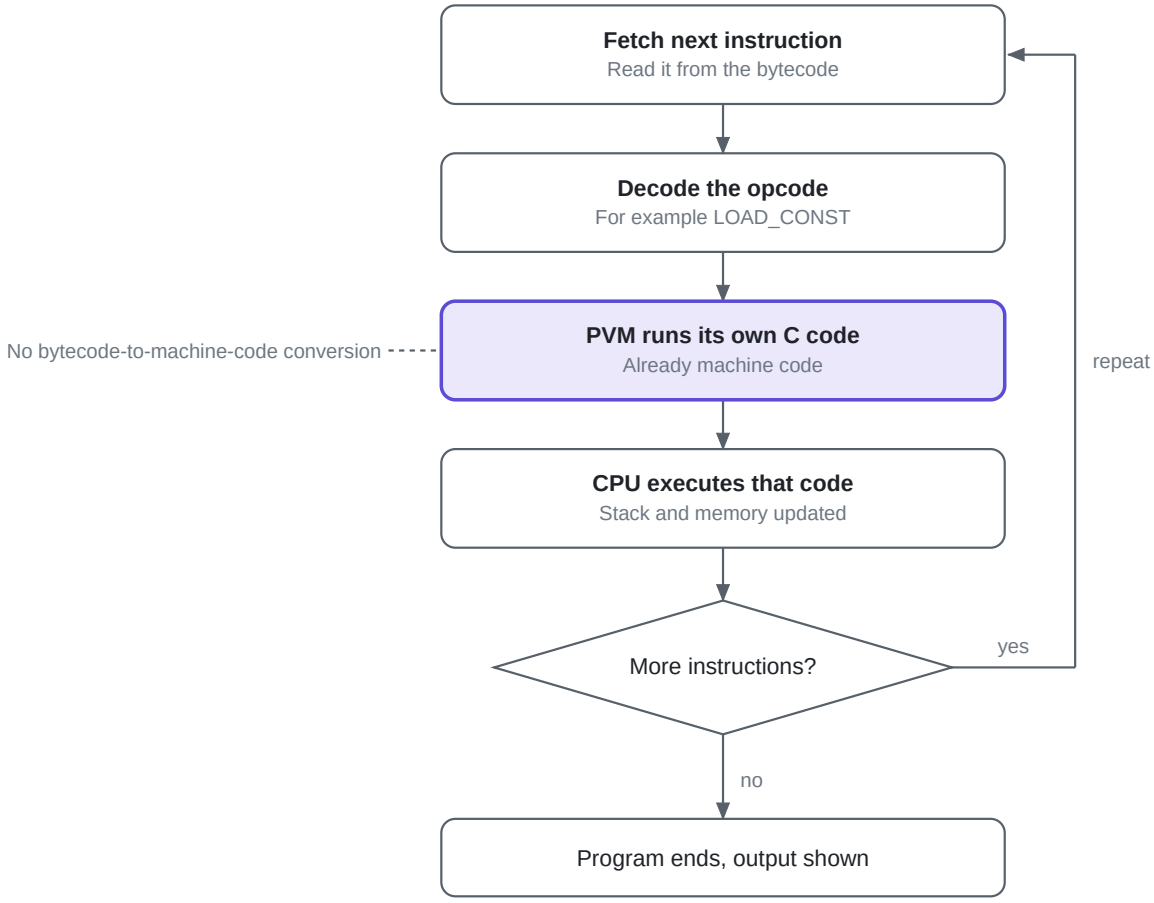
- The PVM is already machine code, because the interpreter is written in C and compiled.
- The PVM reads each bytecode instruction as data and runs the matching C function (for example, `LOAD_CONST` runs a function that pushes a value onto the stack).
- The CPU executes that C function's machine code.
- Analogy: a bytecode instruction is an order in a restaurant. The PVM is a chef who already knows every dish and cooks each order as it is read.
- Exception: JIT-based implementations like PyPy convert frequently used bytecode into machine code while the program runs. Standard CPython does not do this by default.

CPython	PVM
The whole Python interpreter program	One part inside CPython
Includes the compiler, the PVM and the standard libraries	Only runs bytecode
Compiles .py into bytecode	Does not compile anything
What you install and run (python.exe)	Not installed separately

The "C" in CPython stands for the C programming language. Other implementations: PyPy (faster, uses a JIT), Jython (runs on Java's JVM), IronPython (for .NET).

Flowchart 2: how the PVM runs bytecode

The PVM runs its own machine code for every bytecode instruction



The loop repeats for every instruction. The machine code the CPU runs belongs to the PVM itself, so your bytecode is never converted.

Viewing bytecode with dis

`dis` disassembles code, meaning it shows the real bytecode instructions. It does not decompile. `dis.dis(module)` shows only the functions and classes inside a module, so a module with only a top-level `print(...)` shows nothing.

Fix 1: put the code in a function.

```
# hello.py
def greet():
    print("Hello World")

# Demo.py
import hello
import dis
dis.dis(hello.greet)
```

Fix 2: disassemble the whole file's top-level code.

```
import dis
with open("hello.py") as f:
    code = compile(f.read(), "hello.py", "exec")
dis.dis(code)
```

Fix 3: use the command line.

```
python -m dis hello.py
```

Bytecode vs decompiled code

Bytecode is what the virtual machine executes; decompiled code is what a tool rebuilds to help you understand that bytecode (Java example).

Bytecode	Decompiled code
Virtual machine instructions	Source-like statements
Executed by the virtual machine (JVM)	Meant for humans to read
Stored in the .class file	Made by a decompiler when you view the file
The real thing	An approximation of the original source

A decompiler cannot always recover original variable names, comments or formatting.

Interview questions

Question	Short answer
Why is only an imported module's bytecode saved?	Python compiles every file in memory but saves a .pyc only for imports. Caching belongs to the import system, and modules are loaded repeatedly. The main script is small, so caching it saves almost nothing.
Does a .pyc make the program run faster?	No. It makes the program load faster, because compiling is skipped.
Is the .pyc saved every time?	No. It is saved once and reused until the source file changes.
Can you create a .pyc for a script?	Yes, with python -m py_compile script.py. If the script is imported, it is cached like any module.
Did the developers have a specific reason?	The docs state the behavior but not their exact reasoning. The practical reason is that caching helps code that is loaded repeatedly.
Who converts bytecode to machine code?	In CPython, nobody. The PVM, which is already machine code, reads and executes each instruction. Only JIT-based implementations like PyPy convert it at run time.
What is the difference between CPython and the PVM?	CPython is the whole interpreter (compiler, PVM, libraries). The PVM is the part that executes bytecode.

Quick revision

- Python compiles source (.py) into bytecode, then the PVM executes it.
- Bytecode is made for every file that runs. Saving it is a separate step.
- Imported module: .pyc saved in __pycache__, reused while the source time and size are unchanged.
- Main script: compiled in memory, not saved.
- Reason: caching belongs to import, the script is usually small, modules are loaded repeatedly.
- .pyc = faster loading, not faster running.
- The PVM executes bytecode and the CPU executes the PVM. CPython does not convert bytecode to machine code.
- CPython = the standard Python, written in C. The PVM = the bytecode-running part inside it.
- dis.dis(module) shows only functions. Use python -m dis file.py for top-level code.